

Zelig Styler SDK — iOS Integration Guide

This guide covers embedding the Zelig Styler widget into a third-party iOS app. Shoppers build looks, try on outfits, and add items to cart; your app handles dismissal and cart operations through a small delegate / closure bridge.

The SDK ships as a **pre-built closed-source binary** (ZeligStylerSDK.xcframework) — no source code is included.

Prerequisites

- iOS 13.0+
- Xcode 14+ (Xcode 15+ requires one extra build setting for CocoaPods — see Option B, step 4)
- Swift 5.7+
- Your **storeId** and **storeKey** — provided by your Zelig solutions engineer

Release artifacts

Current version: 2.0.1 — the URLs below are pinned to it. When Zelig publishes a newer native SDK, bump these version strings (see Step 10).

Artifact	URL
XCFramework zip (Option A)	https://storage.googleapis.com/ios-widget-sdk-zelig-prod/ZeligStylerSDK-2.0.1.xcframework.zip
CocoaPods podspec (Option B)	https://raw.githubusercontent.com/wearezelig/ZeligStylerSDK-binary/2.0.1/ZeligStylerSDK.podspec
SPM binary package (Option C)	repo https://github.com/wearezelig/ZeligStylerSDK-binary.git — pin by version/tag (2.0.1)

All three channels download the **same** prod XCFramework from GCS. Prefer pinning a **tag / version** (shown above), not **main**, so your app store build stays reproducible.

Step 1 — Install the SDK (pick one)

All three options deliver the same pre-built binary.

Use only one at a time. Before switching, fully remove the previous integration (dragged-in framework, pod entries, or SPM package), otherwise you'll see `Missing package product` or `duplicate-module` errors.

Option A — Direct XCFramework (simplest, no tooling required)

1. Download and unzip:

```
https://storage.googleapis.com/ios-widget-sdk-zelig-prod/  
ZeligStylerSDK-2.0.1.xcframework.zip
```

2. Drag `ZeligStylerSDK.xcframework` into your Xcode project. In the dialog that appears, make sure your **app target** is checked. Either “Reference files in place” or “Copy” works.
3. In your target's **General** → **Frameworks, Libraries, and Embedded Content**, set `ZeligStylerSDK.xcframework` to **Embed & Sign**. (Xcode may default to “Do Not Embed” — you must change it.)

That's it — no package manager involved.

This option has no automatic version management or checksum verification. Upgrading means downloading the **new versioned zip URL** and swapping the file. For SHA-256-verified pinned installs, prefer Option B or C.

Upgrading (example → 2.0.1): download .../
`ZeligStylerSDK-2.0.1.xcframework.zip` → remove the old
`ZeligStylerSDK.xcframework` from the project navigator → drag in the new one
→ confirm **Embed & Sign** → rebuild and ship.

Option A — Direct XCFramework
Option A — Direct XCFramework

Option B — CocoaPods

1. Add one line to your Podfile:

```
platform :ios, '13.0'  
use_frameworks!  
  
target 'YourApp' do  
  # Pin a tagged release (recommended – keep builds  
  # reproducible):  
  pod 'ZeligStylerSDK', :podspec => 'https://  
    raw.githubusercontent.com/wearezelig/ZeligStylerSDK-  
    binary/2.0.1/ZeligStylerSDK.podspec'  
end
```

CocoaPods fetches the podspec from the binary repo tag (GitHub raw); the podspec downloads the pre-built zip from GCS and verifies SHA-256.

2. Run:

```
pod install
```

A successful install prints Installing ZeligStylerSDK (2.0.1).

3. **Always open YourApp.xcworkspace from this point on, not .xcodproj.**

Opening the wrong file causes “no such module ‘ZeligStylerSDK’” at compile time.

4. **Required on Xcode 15+:** target → Build Settings → search User Script Sandboxing → set to **No** (both Debug and Release). Skipping this causes Command PhaseScriptExecution failed with a nonzero exit code — Xcode 15’s sandbox blocks CocoaPods’ framework-embed script on every pod-based project.

Upgrading (example → 2.0.1): change the tag in the Podfile URL, then reinstall:

```
pod 'ZeligStylerSDK', :podspec => 'https://  
raw.githubusercontent.com/wearezelig/ZeligStylerSDK-  
binary/2.0.1/ZeligStylerSDK.podspec'
```

```
rm -rf Pods Podfile.lock  
pod install
```

Installing ZeligStylerSDK (2.0.1) in the output confirms success. Rebuild and ship.

Changing only Podfile.lock / re-running pod install **without** bumping the tag URL will keep you on 2.0.1.

Option B — CocoaPods
Option B — CocoaPods

Option C — Swift Package Manager

The SDK ships as a binary Swift package. Pin it by **version**, not branch.

1. Xcode: **File** → **Add Package Dependencies...**

2. Paste the **git repo URL** — not the zip URL. (Pasting a zip URL prompts for a username/password; Xcode downloads and SHA-256-verifies the zip automatically based on the package’s declaration.)

```
https://github.com/wearezelig/ZeligStylerSDK-binary.git
```

3. Set the **Dependency Rule** to **Up to Next Major Version**, starting at 2.0.1.

That locks you to the 1.x line (safe under semver) without picking up a breaking 2.0.1 automatically.

Use **Exact Version 2.0.1** only if your release process requires freezing a single binary until an explicit bump.

4. **Add to Project** → select your app project → **Add Package**. In the sheet that appears, **Add to Target** → select your app target → **Add**.
5. Verify: the Project Navigator shows **zeligstylersdk-binary**, and your target's **General** → **Frameworks, Libraries, and Embedded Content** lists ZeligStylerSDK.

Upgrading within 1.x (recommended rule = Up to Next Major):

1. Zelig tags a new release on ZeligStylerSDK-binary (e.g. 2.0.1) and publishes a new GCS zip.
2. Xcode → **File** → **Packages** → **Update to Latest Package Versions** (or **Resolve Package Versions**). If Xcode won't refresh: **File** → **Packages** → **Reset Package Caches**, then update again.
3. Confirm the resolved version in the Project navigator / Package.resolved is the new tag.
4. Rebuild and ship.

Upgrading if you used Exact Version: change the rule to the new tag (e.g. 2.0.1), resolve packages, rebuild, and ship.

Option C — Swift Package Manager
Option C — Swift Package Manager

Step 2 — Import

```
import ZeligStylerSDK
```

If this compiles, the installation succeeded. If you see `Unable to resolve module`: Option A — check target membership and the Embed setting; Option B — make sure you opened the `.xcworkspace`.

Step 3 — Store credentials

Your Zelig solutions engineer will provide your **storeId** and **storeKey**.

Security: treat `storeKey` like a password. Do not commit it to source control or embed it as a string literal in your binary. Load it from a secrets manager, a remote config endpoint, or your app's secure configuration layer.

Code examples in this guide use a placeholder store; substitute your own credentials.

Step 4 — Basic integration

Integration Note: The examples below show adding ZeligStyler to a product page, but you can integrate it anywhere in your app — product detail pages, cart, home recommendations, or a dedicated “Stylist” entry point. Replace class/view names with your actual implementation.

SwiftUI (recommended)

```
import SwiftUI
import ZeligStylerSDK

// Example: Add a "Build a Look" button to your existing view
struct ProductDetailView: View {
    let productCode: String // The product SKU/code to open on –
                           // passed from your product page
    let userId: String?     // Current logged-in user ID from
                           // your auth system, or nil for guests
    @State private var showStyler = false

    var body: some View {
        Button("Build a Look") { showStyler = true }
            .sheet(isPresented: $showStyler) {
                ZeligStylerView(
                    config: ZeligStylerConfig(
                        storeId: "YOUR_STORE_ID", //
                        // Replace with your storeId from Zelig
                        storeKey: "YOUR_STORE_KEY", //
                        // Replace with your storeKey from Zelig
                        productCode: productCode,
                        userId: userId
                    ),
                    onAddToCart: { item, completion in
                        // `item` carries every field the SDK
                        // parsed from the widget –
                        // pass them all through, then resolve the
                        // widget's pending state.
                        cart.add(
                            sku: item.sku, //
                            // normalized product SKU
                            size: item.size, //
                            // selected size ("all" for one-size)
                            quantity: item.quantity ??
                                1, // Int? – default to 1 when absent
                            fromZelig: item.fromZelig ?? true //
                                // Bool? – Zelig-originated add
                        ) { success in
                            completion(success)
                        }
                    },
                    onClose: { showStyler = false },
                    onError: { error in
                        print("Zelig error: \(error)")
                    }
                )
            }
            .ignoresSafeArea()
    }
}
```

UIKit

```
import UIKit
import ZeligStylerSDK

// Example: Add a "Build a Look" button to your existing view
// controller
final class ProductViewController: UIViewController,
    ZeligStylerDelegate {
    private var styler: ZeligStyler?

    func showStyler(productCode: String, userId: String? = nil) {
        let config = ZeligStylerConfig(
            storeId: "YOUR_STORE_ID",           // Replace with your
            // storeId from Zelig
            storeKey: "YOUR_STORE_KEY",        // Replace with your
            // storeKey from Zelig
            productCode: productCode,
            userId: userId                     // Pass logged-in
            // user ID, or nil for guests
        )
        let styler = ZeligStyler(config: config, delegate: self)
        self.styler = styler                 // retain the reference
        styler.present(from: self)
    }

    func zeligStyler(_ styler: ZeligStyler,
        didRequestAddToCart item: ZeligCartItem,
        completion: @escaping (Bool) -> Void) {
        // `item` carries every field the SDK parsed from the
        // widget – pass them all through.
        cart.add(
            sku: item.sku,                     // normalized product
            // SKU
            size:
            item.size,                         // selected size ("all" for one-
            // size)
            quantity: item.quantity ?? 1,      // Int? – default to 1
            // when absent
            fromZelig: item.fromZelig ?? true  // Bool? – Zelig-
            // originated add
        ) { success in
            completion(success)
        }
    }

    // Optional – defaults to dismissing the styler.
    func zeligStylerDidRequestClose(_ styler: ZeligStyler) {
        styler.dismiss(animated: true)
    }

    // Optional – called on load or initialization failures.
    func zeligStyler(_ styler: ZeligStyler, didFailWith error:
        ZeligStylerError) {
    }
}
```

```

        print("Zelig error: \$(error)")
    }
}

```

Step 5 — The add-to-cart bridge

When a shopper taps **Add to Bag** in the widget, the SDK delivers a `ZeligCartItem` to your callback. **You must call `completion(true)` on success or `completion(false)` on failure** — the widget updates its in-modal UI from this signal. Respond promptly: the widget has a ~2–4 s fallback timeout, after which it resolves the pending state automatically. If that happens before your callback fires, the widget’s displayed state may fall out of sync with your cart.

SwiftUI — the `onAddToCart` closure:

```

onAddToCart: { item, completion in
    // ZeligCartItem fields:
    // item.sku        - normalized product SKU
    // item.size       - selected size ("all" for one-size)
    // item.quantity   - Int? (nil if the widget didn't supply
    //                   one)
    // item.fromZelig  - Bool? (true = Zelig-originated add-to-
    //                   cart)
    cartService.add(
        sku: item.sku,
        size: item.size,
        quantity: item.quantity ?? 1,
        fromZelig: item.fromZelig ?? true
    ) { result in
        switch result {
        case .success: completion(true)
        case .failure: completion(false)
        }
    }
}

```

UIKit — the `ZeligStylerDelegate` method:

```

func zeligStyler(_ styler: ZeligStyler,
                 didRequestAddToCart item: ZeligCartItem,
                 completion: @escaping (Bool) -> Void) {
    // ZeligCartItem fields:
    // item.sku        - normalized product SKU
    // item.size       - selected size ("all" for one-size)
    // item.quantity   - Int? (nil if the widget didn't supply
    //                   one)
    // item.fromZelig  - Bool? (true = Zelig-originated add-to-
    //                   cart)
    cartService.add(
        sku: item.sku,
        size: item.size,
        quantity: item.quantity ?? 1,
        fromZelig: item.fromZelig ?? true
    )
}

```

```

    ) { result in
        switch result {
        case .success: completion(true)
        case .failure: completion(false)
        }
    }
}

```

ZeligCartItem fields:

Field	Description
sku	Normalized product SKU (from the widget's code / productCode).
size	Selected size string ("all" for one-size items).
quantity	Quantity if the widget supplied one; otherwise nil.
fromZelig	true when the widget flagged this as a Zelig-originated add-to-cart; nil if absent.

Step 6 — Configuration reference

Parameter	Type	Required	Default	Description
storeId	String	Yes	—	Your Zelig store identifier.
storeKey	String	Yes	—	Your store API secret, paired with storeId.
productCode	String	Yes for .productDetail	—	Hero SKU to open on. Pass "" for .buildALook /.myClo
userId	String?	No	nil	Your app's logged-in user ID. See Step 7.
entry	ZeligStylerEntry	No	.productDetail	Which experience to open. See “Entry modes” below.

Entry modes

```

public enum ZeligStylerEntry { case productDetail, buildALook, myCloset }

```

- **.productDetail** (*default*) — open on a specific product. productCode must be non-empty.

- **.buildALook** — open the Build a Look experience with no host-provided product. The widget uses the shopper's last look or the store's default recommendation. Pass `productCode: ""`.
- **.myCloset** — same as `.buildALook`, but lands directly on the My Closet tab.

// Open on a specific product (e.g. from a PDP):

```
ZeligStylerConfig(
  storeId: "YOUR_STORE_ID",
  storeKey: "YOUR_STORE_KEY",
  productCode: "PRODUCT_123"
)
```

// Entry points with no specific product:

```
ZeligStylerConfig(
  storeId: "YOUR_STORE_ID",
  storeKey: "YOUR_STORE_KEY",
  productCode: "",
  entry: .buildALook
)
```

```
ZeligStylerConfig(
  storeId: "YOUR_STORE_ID",
  storeKey: "YOUR_STORE_KEY",
  productCode: "",
  userId: "USER_123", // Pass user ID for authenticated users
  entry: .myCloset
)
```

Step 7 — Shopper identity & closet

`userId` tells the Zelig backend how to scope the shopper's closet:

- **Non-nil (signed in)** — the closet is tied to this user and syncs across devices. An anonymous closet is automatically inherited on first sign-in.
- **nil (guest)** — a device-local anonymous closet is used instead.

```
let config = ZeligStylerConfig(
  storeId: "YOUR_STORE_ID", // Replace with your storeId
                        from Zelig
  storeKey: "YOUR_STORE_KEY", // Replace with your storeKey
                        from Zelig
  productCode: productCode,
  userId: userId // Pass your logged-in user ID,
                or nil for guests
)
```

Step 8 — Error handling

Implement the optional error callback to surface load or initialization failures:

```
// SwiftUI – onError closure (see Step 4)
// UIKit – ZeligStylerDelegate method:
func zeligStyler(_ styler: ZeligStyler, didFailWith error:
    ZeligStylerError) {
    // Possible values:
    // .loaderLoadFailed(URL)
    // .widgetInitTimeout
    // .modalMountTimeout
    // .webViewNavigationFailed(underlying: Error)
    // .hostPageUnavailable
}
```

Your responsibilities:

1. Always call the add-to-cart completion — on both success and failure.
2. Ensure the device has network access before presenting the styler.

Step 9 — Testing & debugging

Use the SKUs provided by your Zelig solutions engineer for your store.

In debug builds, connect **Safari** → **Develop** → **(your device or simulator)** → **Zelig Styler** to inspect the embedded WKWebView. The SDK forwards JavaScript console.log / console.error output to the Xcode console — filter for [Zelig iOS] to isolate SDK messages.

Step 10 — SDK versions & updates (how Revolve / hosts upgrade)

Two layers update independently:

Layer	What it is	How it updates
Widget (JS)	Styler UI, looks, algorithms	Loaded at runtime from Zelig’s CDN. No app store release required — shoppers get the latest widget the next time they open the styler.
Native SDK	ZeligStylerSDK.xcframework bridge	Version-pinned in your Xcode project. You only bump it when Zelig ships a new native release (bridge / public API). Your solutions engineer notifies you in advance.

Pinning 2.0.1 is intentional: your App Store binary stays reproducible until you choose to upgrade the native SDK.

When Zelig publishes a new native version (e.g. 2.0.1 → 2.0.1)

Zelig will:

1. Upload ZeligStylerSDK-2.0.1.xcframework.zip to GCS (ios-widget-sdk-zelig-prod).
2. Tag <https://github.com/wearezelig/ZeligStylerSDK-binary> as 2.0.1 (Package.swift + podspec updated with the new URL + SHA-256).
3. Tell you the new version number.

Then upgrade with **your** install method only:

Your install	What you change	Then
A — Direct XCFramework	Download the new versioned zip URL (.../ZeligStylerSDK-2.0.1.xcframework.zip)	Remove old framework → drag in new → Embed & Sign → rebuild → ship rm -rf Pods Podfile.lock && pod install → confirm Installing ZeligStylerSDK (2.0.1) → rebuild → ship File → Packages → Update to Latest Package Versions → confirm
B — CocoaPods	Bump the tag in the Podfile URL (.../ZeligStylerSDK-binary/2.0.1/ZeligStylerSDK.podspec)	Package.resolved → rebuild → ship. If you used Exact Version , change it to 2.0.1 first.
C — SPM	If rule is Up to Next Major from 2.0.1: nothing in Project settings	

Previous version zips remain online — old app builds keep resolving.

The SDK follows semantic versioning. Any **1.x** update is backward-compatible with your existing integration code. A future **2.x** would be a breaking change and would be called out explicitly (SPM's "Up to Next Major" would not auto-adopt it).

Step 11 — Release checklist

☐

Your storeId / storeKey are correct.

☐

storeKey not committed to source control.

☐

productCode passed and present in your store's catalog.



Add-to-cart callback implemented; completion called on both success and failure.



Logged-in `userId` passed for authenticated shoppers (`nil` only for guests).



Tested on multiple device sizes (phone and tablet).

Troubleshooting

Symptom	Fix
Missing package product 'ZeligStylerSDK'	A previous install method wasn't fully removed. Check Package Dependencies, the Frameworks list, and the Podfile — keep exactly one.
Command PhaseScriptExecution failed (CocoaPods)	Xcode 15+ sandbox restriction. Build Settings → User Script Sandboxing → No (see Option B, step 4).
Unable to resolve module 'ZeligStylerSDK'	Option B: you opened <code>.xcodeproj</code> — open <code>.xcworkspace</code> instead. Option A: check target membership and the Embed & Sign setting.
Widget blank or not loading	Check network connectivity; verify <code>storeId</code> / <code>storeKey</code> ; open Safari Web Inspector and filter for [Zelig iOS] logs.
Add to cart hangs or never resolves	Confirm your <code>onAddToCart</code> / <code>delegate</code> method is wired up and that <code>completion(true/false)</code> is called on every code path.
Wrong product opens	Verify <code>productCode</code> is correct and exists in your store's catalog.
Touches not registering on a physical device	Contact your Zelig solutions engineer — this is typically a host-app layout configuration issue.

API reference

```
public enum ZeligStylerEntry { case productDetail, buildALook, myCloset }
```

```
public struct ZeligStylerConfig {  
    public let storeId: String  
    public let storeKey: String  
    public let productCode: String  
    public let userId: String?  
    public let entry: ZeligStylerEntry  
  
    public init(  
        storeId: String,  
        storeKey: String,  
        productCode: String,  
        userId: String? = nil,  
        entry: ZeligStylerEntry = .productDetail  
    )  
}
```

```
public struct ZeligCartItem {  
    public let sku: String  
    public let size: String  
    public let quantity: Int?  
    public let fromZelig: Bool?  
}
```

```
public enum ZeligStylerError: Error {  
    case loaderLoadFailed(URL)  
    case widgetInitTimeout  
    case modalMountTimeout  
    case webViewNavigationFailed(underlying: Error)  
    case hostPageUnavailable  
}
```

```
public protocol ZeligStylerDelegate: AnyObject {  
    // Required:  
    func zeligStyler(_ styler: ZeligStyler,  
                    didRequestAddToCart item: ZeligCartItem,  
                    completion: @escaping (Bool) -> Void)  
  
    // Optional:  
    func zeligStylerDidRequestClose(_ styler: ZeligStyler)  
    func zeligStyler(_ styler: ZeligStyler, didFailWith error: ZeligStylerError)  
}
```

```
public final class ZeligStyler {  
    public init(config: ZeligStylerConfig, delegate: ZeligStylerDelegate? = nil)  
    public func present(from presenter: UIViewController, animated: Bool = true)  
    public func dismiss(animated: Bool = true)
```

```

}

public struct ZeligStylerView: UIViewControllerRepresentable {
    public init(
        config: ZeligStylerConfig,
        onAddToCart: @escaping (ZeligCartItem, @escaping (Bool)
        -> Void) -> Void,
        onClose: (() -> Void)? = nil,
        onError: ((ZeligStylerError) -> Void)? = nil
    )
}

```

Support

Contact your Zelig solutions engineer for store credentials, environment configuration, and any merchant-specific setup.

Version: 2.0.1 · **Last updated:** July 2026